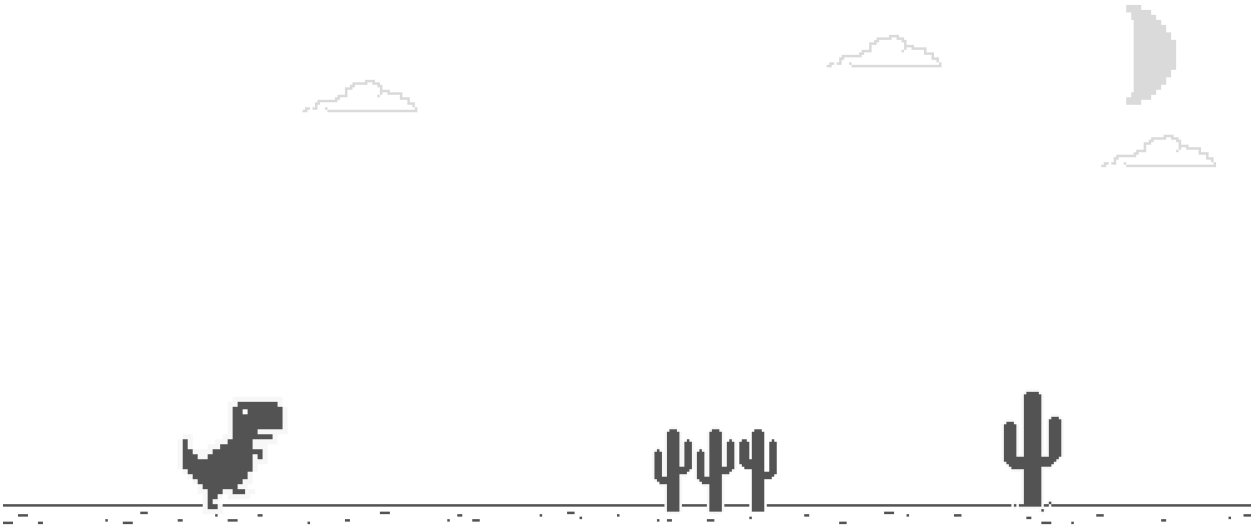
# THE DINO GAME

You've seen it a hundred times: the Wi-Fi drops, Chrome shrugs, and a little pixelated T-Rex appears, ready to sprint through a desert the moment you hit the spacebar.

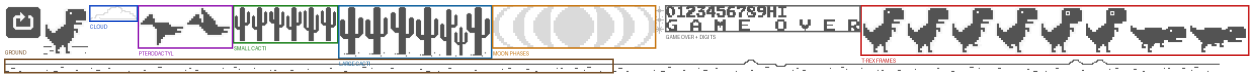
That tiny game, hidden behind the "No Internet" error since 2014, is played roughly 270 million times every month. Its internal codename at Google was "Project Bolan," a nod to Marc Bolan. And because Chrome is built on the open-source Chromium project, the entire game – every constant, design decision, and hack – is sitting in public for anyone to read.



In this lab assignment, we'll walk through the real source code and unpack how the dino game actually works. If you want the game open in a tab while you read, you don't need to kill your Wi-Fi. You can play the Dinosaur Game online.

The typescript code itself lives in the Chromium tree, browsable at source.chromium.org. This is historically a single file of roughly 3,000 lines of dependency-free vanilla JavaScript, drawing on a plain `<canvas>`.

By the way, here's the entire game's artwork. It's one small PNG, with every sprite the code refers to by coordinates.




**The Clock: Why the Game Runs the Same Speed Everywhere**

The first problem every game has to solve: browsers don't repaint at a fixed rate. A 144 Hz gaming monitor fires `requestAnimationFrame` 144 times a second. But a struggling laptop might manage 40. If you move things a fixed number of pixels per frame, your game literally runs 3× faster on better hardware.

The dino's solution is the standard one, executed cleanly. The game defines its speeds in pixels per frame *at an assumed 60 FPS*, then scales every movement by how much time passed:

```
this.msPerFrame = 1000 / FPS;
```

```
// ...in each update:
this.xPos -= Math.floor((currentSpeed * FPS / 1000) *
deltaTime);
```

Every moving thing in the game from the dino's jump to the cacti, clouds, and even the moon, is multiplied by `deltaTime`. That's why your high score is comparable to your friend's, whatever machines you're both on.

If you take one engineering habit away from this assignment, take this one: **never move anything by "per frame" amounts. Always scale by elapsed time.**

**The T-Rex: Four Constants and a Typo**

The dino's entire physical existence is defined by a handful of numbers in `Trex.config` and `Runner.config`:

```
GRAVITY: 0.6,
INIITAL_JUMP_VELOCITY: -10,
SPEED: 6,
ACCELERATION: 0.001,
MAX_SPEED: 13,
```

Yes, you read that right: `INIITAL_JUMP_VELOCITY`, with three I's. That misspelling shipped in Chrome, on billions of devices, and has survived for roughly a decade, because renaming it was never worth the risk. Let it comfort you the next time you find a typo in your own production code.

The units are pixels per frame at 60 FPS. Convert them and the physics becomes intuitive: gravity is 0.6 px/frame², jump velocity –10 px/frame. Run the math and the jump arc peaks at about 83 pixels roughly 0.28 seconds after takeoff. On a 150-pixel-tall playfield, that's more than half the screen.

The world starts scrolling at 6 px/frame (360 px/s) and gains 0.001 px/frame every frame until it hits the cap of 13, a little over twice the starting speed. That cap matters: it's the promise that the game gets *hard*, but never *impossible*.

But here's the detail most clones miss: **the jump height is variable.** Watch the code that runs when you release the spacebar:

```
endJump: function () {
    if (this.reachedMinHeight &&
        this.jumpVelocity < this.config.DROP_VELOCITY) {
        this.jumpVelocity = this.config.DROP_VELOCITY;
    }
},
```

Tap the spacebar and the dino does a short hop, but hold it and the dino rides the full arc. Releasing the key early clamps the upward velocity, cutting the jump short (as long as a minimum height was reached, so you can't glitch yourself into a cactus). And if you press the Down arrow mid-air, `setSpeedDrop` multiplies the fall speed by 3, slamming the dino back to the ground for a fast recovery.

Two tiny mechanics, and suddenly the single-button game has an expressive skill ceiling: short hop, full jump, fast slam. That's why the top players' runs look nothing like yours initially.

**The Fairness Engine: How the Game Refuses to Cheat You**

This is my favorite part of the codebase, because none of it is visible. You can only *feel* it. Every obstacle in the game is declared with a small config, and the configs encode a set of fairness rules. Here's the small cactus:

```
{
    type: 'CACTUS_SMALL',
    width: 17,
    height: 35,
    multipleSpeed: 4,
    minGap: 120,
    minSpeed: 0,
    // ...
}
```

Let's unpack the rules hiding in there and elsewhere in the spawning code:

**Rule 1: The first three seconds are empty.** `CLEAR_TIME: 3000` guarantees no obstacle spawns for the first three seconds of a run. That's silent onboarding: you get a moment to feel the controls before the game asks anything of you.

**Rule 2: Clusters are gated by speed.** A cactus can spawn as a group of up to `MAX_OBSTACLE_LENGTH: 3`, but only when the current speed exceeds its `multipleSpeed` (4 for small cacti, 7 for large). Why? Because your jump *distance* grows with the world speed. The arc lasts a fixed time, so the faster the ground moves, the more ground you clear per jump.

Wide obstacles only appear once your jump is physically long enough to clear them. The game never generates a wall it knows you can't cross.

**Rule 3: Gaps scale with speed too.** The gap after each obstacle is computed as roughly `obstacleWidth × speed + minGap × 0.6`, plus randomness. So the faster the game, the more room you're given to react. Difficulty comes from the speed itself, never from unfair spacing.

**Rule 4: No obstacle appears three times in a row.** There's a function whose entire job is variety:

```
duplicateObstacleCheck: function (nextObstacleType) {
    var duplicateCount = 0;
    for (var i = 0; i < this.obstacleHistory.length; i++) {
        duplicateCount = this.obstacleHistory[i] ==
nextObstacleType ?
            duplicateCount + 1 : 0;
    }
    return duplicateCount >=
Runner.config.MAX_OBSTACLE_DUPLICATION;
},
```

With `MAX_OBSTACLE_DUPLICATION: 2`, the spawner keeps a history and re-rolls if the same obstacle type would appear a third consecutive time. You've never noticed this rule, which is exactly the point. You'd have noticed its absence.

**Rule 5: The pterodactyl is a late-game boss.** Its config says `minSpeed: 8.5`, meaning it can't appear at all until you're two-thirds of the way to max speed. It flies at one of three heights (so sometimes you jump it, sometimes you duck, and sometimes at 50 pixels you must decide), it never spawns in groups (`multipleSpeed: 999`), and it has its own `speedOffset: 0.8`, meaning each pterodactyl flies slightly faster or slower than the world scrolls. That last detail breaks your rhythm-based muscle memory precisely when you've gotten comfortable.

Together these rules are the answer to a question every game designer faces: how do you make a game *harder* without making it *unfair*? Players can feel the difference between "I lost because I was slow" and "I lost because the game cheated". And the dino, in ten years and trillions of runs, has never cheated anyone.

**Collision Detection: The Dino Is Six Rectangles**

Naïve collision detection would wrap the dino in one bounding box and check overlap. But look at the dino: he has a snout sticking out, a tail, a gap under his chin. With a single box, a cactus grazing the empty air under his jaw would kill you, and it would feel terrible.

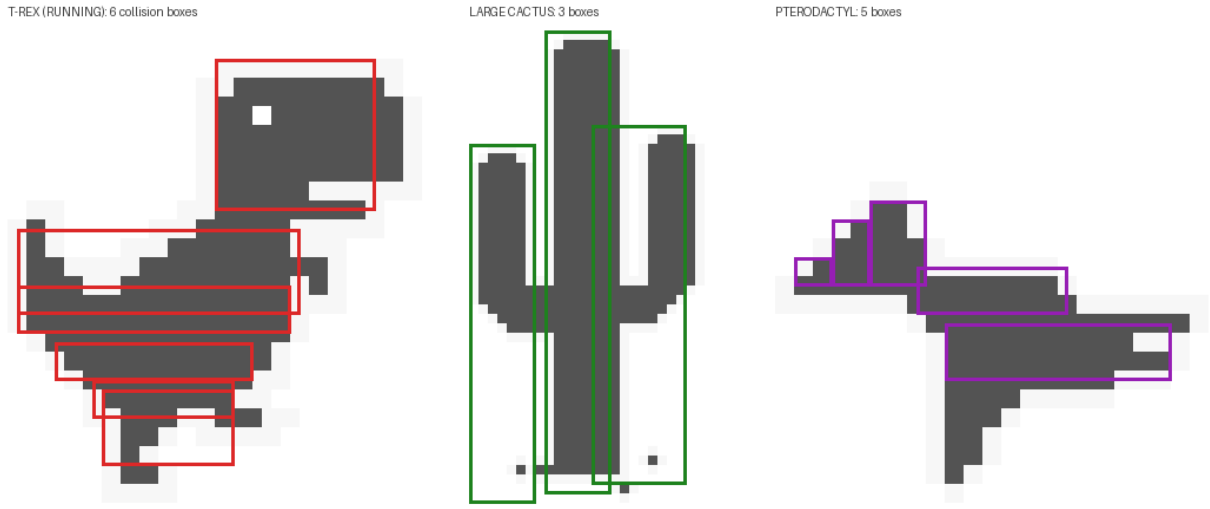
So the real dino is six boxes:

```
Trex.collisionBoxes = {
    RUNNING: [
        new CollisionBox(22, 0, 17, 16),   // head
        new CollisionBox(1, 18, 30, 9),    // torso
        new CollisionBox(10, 35, 14, 8),   // legs
        new CollisionBox(1, 24, 29, 5),
        new CollisionBox(5, 30, 21, 4),
        new CollisionBox(9, 34, 15, 4)
    ],
    DUCKING: [
        new CollisionBox(1, 18, 55, 25)    // one long low
box
    ]
};
```

Six small rectangles that trace the dino's actual silhouette: head, torso, and a staircase of boxes down the belly and legs. When the dino ducks, the whole set is swapped for one long, low box.

The obstacles get the same treatment: a small cactus is three boxes tracing its trunk and arms, and the pterodactyl is *five*, following its wings and beak.

Here's what those boxes actually look like, drawn over the real sprites:




The algorithm is a classic two-phase check: first a cheap outer-box test (do the overall rectangles even touch?), and only if that passes, does the detailed loop comparing every dino box against every obstacle box occur. Therefore, the test is fast in the common case and precise in the moment that matters. If a cactus needle visually passes through the notch under the dino's chin, you live, and the code agrees with your eyes.

This is the cheapest possible version of a technique that scales all the way up to fighting-game hitboxes and hurtboxes. The lesson generalizes: **collision should match what the player sees, not what's convenient for the math.**

For React, it is a good idea to start here with the source code to help get you started with building the foundation:

https://github.com/Vasu7389/react-project-ideas/tree/master/day006/dino-game

A few things you will notice that are missing, are the bosses, the clouds, the initialization of the game with the space bar, and the mechanics for ducking. All of these can be considered features that you may choose to include in your implementation.

For your assignment, we are looking for 5 new features or improvements for the dino game, here is a list of more suggestions (but feel free to include your own) –

## 🎮 Gameplay Mechanics

- **Dynamic Power-Ups:** Introduce rare items that the dinosaur can jump to collect. Examples include a "shield" to absorb one hit, a "pepper" for a temporary speed boost, or a "pterodactyl wings" power-up to fly briefly.
- **Crouch Attacks:** Allow the dinosaur to hit low-flying pterodactyls by tail-swiping or jumping on top of them, rather than just ducking under them.
- **Variable Biomes:** Transition the background and obstacles seamlessly into new environments as the score increases, moving from the desert to an **ice age** (slippery physics), a **volcanic zone** (falling ash), or a **cyberpunk city**.

## 🎨 Visuals & Customization

- **Unlockable Skins:** Let players use their high scores or collected "egg coins" to unlock new characters. They could play as a **T-Rex**, a **Triceratops**, or even funny outfits like a retro astronaut or a top hat and tie.
- **Weather Effects:** Introduce random, purely visual weather shifts like heavy rain, sandstorms, or a day/night cycle that affects visibility and adds atmosphere.
- **Smoother Animations:** Upgrade the 8-bit aesthetic slightly with fluid, modern pixel art animations for running, jumping, and crashing, while retaining the monochromatic look.
- **Add additional visuals:** Add clouds, a moon, or different expressions for the dino depending on the interactions you specify in the game.

## 🏆 Progression & Community

- **Local & Global Leaderboards:** Save the player's top scores locally on their browser and add an optional global leaderboard to see how they rank against friends or players worldwide.
- **Daily Challenges:** Add a daily mode with specific modifiers—such as "double speed," "no ducking allowed," or "reversed gravity"—to keep the gameplay fresh every day.
- **Achievement System:** Create fun milestones for players to chase, like **"Cactus Clearer"** (jump over 50 cacti in one run) or **"Night Owl"** (survive a full night cycle).

## 🕹️ Quality of Life

- **Boss Encounters:** Every 1,000 points, spawn a giant boss obstacle (like a massive Brachiosaurus blocking the path) that requires a specific sequence of jumps and ducks to pass.